

ML Tasks, Linear Models, Gradient Descent

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

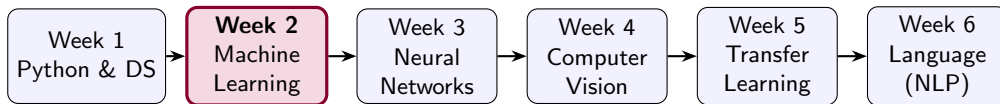


موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 2, Day 08, Lecture

26 September 2026

1. Big Picture and Motivation
2. Learning Outcomes
3. AI, Machine Learning, Deep Learning
4. ML Tasks
5. A Model Is a Function
6. How Good Is a Line? The Loss
7. Gradient Descent
8. Logistic Regression
9. Olympiad Practice
10. Limitations and What Comes Next
11. Summary
12. References



Today: Day 8 of 42

- ▶ **Last week:** Python, NumPy arrays, pandas tables, plots. You classified a flower with an if rule (Day 01) and with the nearest centroid (Days 02 and 04).
- ▶ **Today:** what “machine learning” really means, the kinds of tasks it solves, and the first model that **learns its numbers from data**: a straight line.
- ▶ **This week:** tomorrow, how we score a model and when it cheats (overfitting); Monday, the other classic models (KNN, SVM, trees). Days 11 to 13 go deeper into all of them.

In Day 01 we looked at a plot, picked two numbers by eye and wrote the rule ourselves:

```
def predict(petal_length):  
    if petal_length < 2.5:          # 2.5 chosen by looking at the plot  
        return "Iris-setosa"  
    elif petal_length < 4.9:       # 4.9 chosen the same way  
        return "Iris-versicolor"  
    return "Iris-virginica"
```

Now a new question: **a petal is 4.0 cm long. How wide is it?** By hand we might guess “about a third of the length”:

flower	petal length $\times$	guess 0.30 $\times$	true width	error
row 0	1.4	0.42	0.2	+0.22
row 50	4.7	1.41	1.4	+0.01
row 100	6.0	1.80	2.5	-0.70

The problem

Hand-made rules are guesses, and we cannot tell how far from the best guess we are.

- ▶ Our rule $y = 0.30x$ has a mean squared error of **0.089** on the 150 flowers.
- ▶ The best straight line, $y = 0.416x - 0.363$, has **0.042**: half the error. Nobody finds 0.416 and -0.363 by eye.
- ▶ Two thresholds for one column took us minutes. A real task has 30 columns and 10,000 rows: no human can tune that by looking.

The pattern

Every hand rule is a function with some numbers inside it (2.5, 4.9, 0.30). Picking those numbers is the hard part, and it is exactly what a computer can do for us.

In one sentence

A **model** is a mathematical function with adjustable numbers (its **parameters**); **learning** means letting the computer choose the numbers that make the error on the data as small as possible.

- ▶ The simplest such function is the straight line $y = mx + b$: two parameters, a slope and an intercept.
- ▶ The computer finds them by **gradient descent**: measure the error, see which way it shrinks, step that way, repeat.
- ▶ The same recipe, with a different function, gives **logistic regression** for yes/no questions, and in Week 3 gives neural networks.

Where you will meet it

Almost every tabular Olympiad task says “predict this column”. If the column is a number (price, score, age) it is a regression; if it is a category (species, spam, digit) it is a classification. Today's models are the baseline you build first in both.

By the end of this session, you will be able to:

- ▶ **Explain** what AI, machine learning and deep learning are, and how they sit inside each other.
- ▶ **Recognise** the task type of a problem: regression, classification, clustering, or dimensionality reduction.
- ▶ **Describe** a model as a function with parameters, and read $y = mx + b$ as slope and intercept.
- ▶ **Compute** the mean squared error of a line on a few points by hand and in NumPy.
- ▶ **Run** gradient descent by hand for one step and in code until the loss stops shrinking.
- ▶ **Fit** a linear regression and a logistic regression with scikit-learn and read back their parameters.

ML Tasks: **AI, Machine Learning, Deep Learning**

Definition

Artificial intelligence (AI) is any computer program that does something which needs intelligence when a person does it: recognise a face, translate a sentence, choose a chess move, predict a price.

- ▶ The definition is about the **task**, not about how the program is written.
- ▶ **Two ways to build it.** Write the rules yourself (Day 01's `if petal_length < 2.5`), or let the program find the rules from examples.
- ▶ A 1997 chess computer beat the world champion with rules and search written by people. It is AI, but it is not machine learning.
- ▶ Your phone's face unlock, a spam filter and a translation app are AI too, and all three learned from data.

Definition

Machine learning (ML) is the part of AI where the computer finds the rules itself from examples, instead of a person typing them in.

	Traditional programming	Machine learning
you give	rules + data	data + answers
you get	answers	rules (a model)
Day 01 example	if $x < 2.5$ typed by us, then predictions	150 flowers with their species, then the
who picks the numbers	a person, by eye	the computer, by minimising an error

Intuition

A student who memorises the teacher's rules is traditional programming. A student who sees 150 solved examples and works out the rule is machine learning.

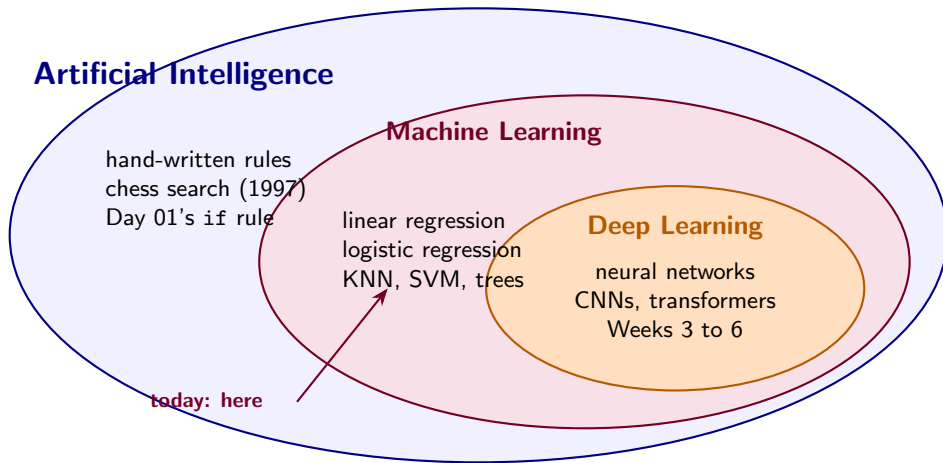
Definition

Deep learning (DL) is machine learning with a **neural network**: a model built from many layers of simple functions stacked on top of each other.

- ▶ “Deep” means many layers. Today’s line has one layer and two parameters; a Week 4 image model has dozens of layers and millions.
- ▶ Classical ML needs a person to choose the input columns (petal length, petal width). Deep learning can start from raw pixels, sound or text and find its own columns.
- ▶ That is why DL wins on images, audio and language (Weeks 3 to 6), while classical ML still wins on most tables (this week).

Skipped on purpose

How a neural network is built and trained is Week 3. Today’s gradient descent is the same training method it uses.



Every deep learning model is a machine learning model, and every machine learning model is AI. The reverse is false.

ML Tasks: **The Kinds of Tasks**

Vocabulary

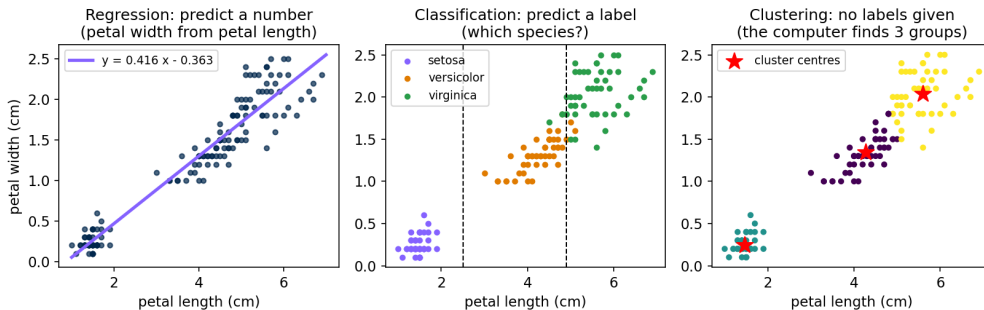
A **sample** is one row. A **feature** is one input column (the matrix X). The **target** (or **label**) is the column we want to predict (the vector y).

row	sepal length	sepal width	petal length	petal width	species
0	5.1	3.5	1.4	0.2	setosa
50	7.0	3.2	4.7	1.4	versicolor
100	6.3	3.3	6.0	2.5	virginica

X : 150 rows $\times$ 4 features y : 150 labels

- ▶ If the table has a target column, the task is **supervised**: the answers are given, the model learns to reproduce them.
- ▶ If there is no target column, the task is **unsupervised**: the model looks for structure in X alone.

Supervised: Regression and Classification



Source: Iris data set (Fisher, 1936), Kaggle uciml/iris; rendered by scripts/day08_ml_figures.py. Left and middle panels.

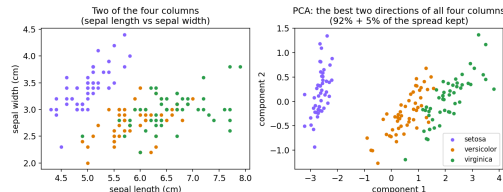
- **Regression:** the target is a **number**. Petal width from petal length. The model is a line; the output can be any value, like 1.30.
- **Classification:** the target is a **category**. The species from the two petal measurements. The model draws boundaries; the output is one of the labels.

Clustering (right panel of the last figure): no labels at all. The algorithm (**K-Means**) is told “find 3 groups” and finds them from the distances alone. On Iris it recovers the three species almost perfectly, without ever seeing the names.

Dimensionality reduction: 4 columns are hard to plot. **PCA** (principal component analysis) finds the two directions along which the flowers spread the most and keeps only those. Here they keep 97% of the spread.

Skipped on purpose

How K-Means and PCA compute their answer is Day 13. Today you only need to recognise the two tasks by name.



Source: Iris, Kaggle uciml/iris; sklearn.decomposition.PCA;
scripts/day08_ml_figures.py.

Task	Target given?	Output	Example	Model and score
Regression	yes, a number	a number	petal width, price	linear regression; MSE, MAE
Binary classification	yes, yes/no	0 or 1	spam or not	logistic regression; accuracy,
Multiclass classification	yes, k labels	one of k	species, digit 0 to 9	logistic, one per class; accur
Clustering	no	a group id per row	customer segments	K-Means (Day 13); no right
Dimensionality reduction	no	fewer columns	4 columns to 2	PCA (Day 13); spread kept

- ▶ The first question in any task: **what is the target column, and is it a number or a category?** That fixes the model family and the metric.
- ▶ Tomorrow (Day 09) is about the last column: MSE, MAE, accuracy, F1 and when each one lies.

Read the task statement, name the task type before touching any code.

1. "Predict the selling price of a flat from its area, floor and district."
2. "Given a 28×28 image, say which digit 0 to 9 it shows."
3. "Given a message, decide whether it is spam."
4. "Here are 5,000 customers with 20 columns and no labels; split them into 4 segments."
5. "Predict tomorrow's temperature from the last 7 days."

Read the task statement, name the task type before touching any code.

1. “Predict the selling price of a flat from its area, floor and district.”
2. “Given a 28×28 image, say which digit 0 to 9 it shows.”
3. “Given a message, decide whether it is spam.”
4. “Here are 5,000 customers with 20 columns and no labels; split them into 4 segments.”
5. “Predict tomorrow’s temperature from the last 7 days.”

Answers

1 **regression** (a price is a number). 2 **multiclass classification**. 3 **binary classification**.
4 **clustering** (no labels). 5 **regression** (a temperature is a number, even if the input is a time series).

ML Tasks: **A Model Is Just a Function**

Definition

A **model** is a function f that takes the features x and returns a prediction $\hat{y} = f(x)$. Inside f sit some numbers, the **parameters**, that we are free to change. Training changes them; predicting uses them.

```
m, b = 0.30, 0.0           # the dials (parameters)

def predict(x):             # the model: a function of x
    return m * x + b        # the numbers inside decide the answer

predict(4.0)                 # 1.2 with these dials
```

- ▶ Same function, different dials, different predictions. Day 04's fit sets the dials; predict reads them.
- ▶ “ $\hat{y}$ ” (y-hat) is the model's guess; “ y ” without the hat is the true value in the table.

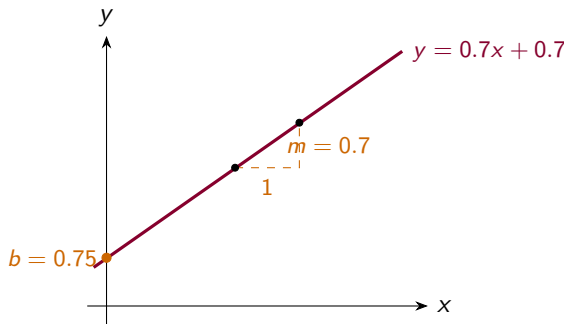
The Linear Equation: $y = mx + b$

$$y = mx + b$$

- ▶ x : the input (petal length).
- ▶ y : the output (petal width).
- ▶ m : the **slope**: how much y changes when x grows by 1.
- ▶ b : the **intercept**: the value of y when $x = 0$, where the line crosses the y axis.

Read it aloud

“Start at height b , then climb m for every step to the right.”

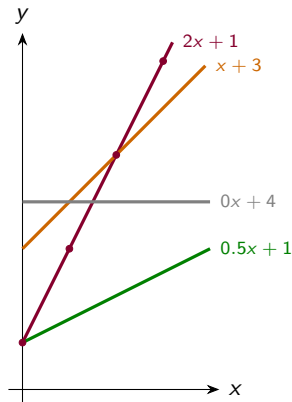


Slope and Intercept, With Numbers

Take $y = 2x + 1$ (slope 2, intercept 1):

x	$y = 2x + 1$	
0	$2 \cdot 0 + 1 = 1$	the intercept
1	$2 \cdot 1 + 1 = 3$	up by 2
2	$2 \cdot 2 + 1 = 5$	up by 2 again
3	$2 \cdot 3 + 1 = 7$	

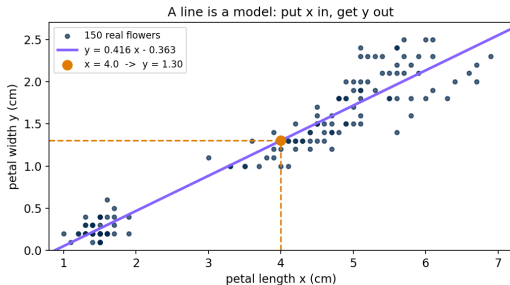
Each step right adds exactly $m = 2$. A negative m makes the line go down; $m = 0$ makes it flat at height b .



Change b : the line shifts up or down. Change m : it tilts. Two dials cover every straight line.

The best line for the 150 flowers is is

$$\hat{y} = 0.416x - 0.363$$



Source: Iris, Kaggle uciml/iris; line fitted with
`sklearn.linear_model.LinearRegression`; scripts/day08_ml_figures.py.

Worked example

Petal length $x = 4.0$: $\hat{y} = 0.416 \cdot 4.0 - 0.363 = 1.664 - 0.363 = 1.30$ cm.

Petal length 1.4 (a setosa): $0.582 - 0.363 = 0.22$; the true width is 0.2.

- ▶ Slope 0.416: each extra centimetre of length adds about 0.42 cm of width.
- ▶ Intercept -0.363 : the line's height at length 0. No flower lives there, so the negative value is harmless.

With d features the line becomes a sum, one slope per feature:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b = \mathbf{w} \cdot \mathbf{x} + b$$

- ▶ $\mathbf{x} = (x_1, \dots, x_d)$: one row of X . $\mathbf{w} = (w_1, \dots, w_d)$: the **weights**, one slope per feature.
 b : the **bias**, the intercept's new name.
- ▶ $\mathbf{w} \cdot \mathbf{x}$ is Day 02's dot product: multiply pairwise, add up.

Worked example with two features

Weights $\mathbf{w} = (0.5, -0.2)$, bias $b = 1$, flower $\mathbf{x} = (4.0, 3.0)$:

$$\hat{y} = 0.5 \cdot 4.0 + (-0.2) \cdot 3.0 + 1 = 2.0 - 0.6 + 1 = 2.4$$

ML people say “weights and bias”; your math book says “slopes and intercept”. Same dials. In NumPy, for a whole table at once: $\mathbf{\hat{y}} = \mathbf{X} @ \mathbf{w} + \mathbf{b}$ (Day 05).

ML Tasks: **How Good Is a Line? The Loss**

Definition

For one sample, the **error** (also called the residual) is the prediction minus the truth: $e = \hat{y} - y$. Positive means the line is too high, negative too low.

A tiny data set we will use all day: three points (1, 3), (2, 5), (3, 7). Try the line $\hat{y} = 1 \cdot x + 1$:

x	y (true)	$\hat{y} = x + 1$	$e = \hat{y} - y$	e^2
1	3	2	-1	1
2	5	3	-2	4
3	7	4	-3	9

- ▶ Three errors, all negative: this line is too low everywhere. We need **one number** for the whole line, not three.
- ▶ Adding the raw errors would let a +2 cancel a -2. Squaring first makes every error count.

$$\text{MSE}(m, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = \frac{1}{n} \sum_{i=1}^n (m x_i + b - y_i)^2$$

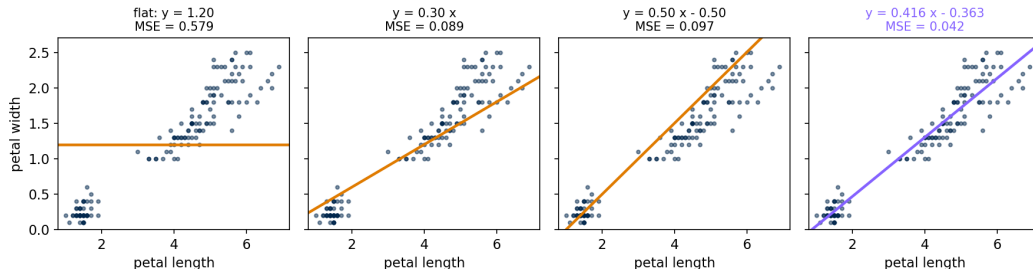
- ▶ n : number of samples; $\hat{y}_i$, y_i : prediction and truth for row i ; $\sum$: add over all rows (Day 02).
- ▶ MSE is a **loss function**: 0 for a perfect model, larger the worse the model is.

Worked example

For the line $\hat{y} = x + 1$ on the three points: $\text{MSE} = \frac{1 + 4 + 9}{3} = \frac{14}{3} = 4.67$. For $\hat{y} = 2x + 1$: predictions 3, 5, 7, errors 0, 0, 0, $\text{MSE} = 0$. That line is perfect.

Skipped on purpose: why squares and not absolute values (MAE), and what outliers do to each.
Tomorrow, Day 09.

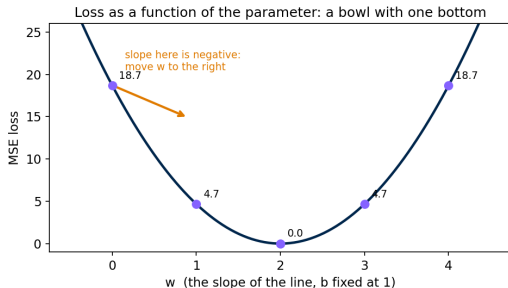
The Loss Picks the Line



Source: Iris, Kaggle uciml/iris; MSE of each candidate computed on the 150 flowers; scripts/day08_ml_figures.py.

- ▶ Four candidate lines, four losses. The flat line (predict the average width for everyone) scores 0.579; our hand guess 0.089; the best line 0.042.
- ▶ **Fitting a model** means: among all possible values of (m, b) , find the pair with the smallest loss.

The Loss Is a Function of the Parameters



Source: the three-point example, b fixed at 1, MSE computed for 200 values of w ; `scripts/day08_ml_figures.py`.

Fix $b = 1$ and let the slope w vary. Each w gives one loss:

w	0	1	2	3	4
MSE	18.7	4.7	0	4.7	18.7

- ▶ The loss draws a **bowl** over the parameter; its bottom is the best w .
- ▶ With (w, b) the bowl is a surface; with a million parameters we cannot draw it, but it still has a bottom.
- ▶ **Learning = walking to the bottom.** The next section is the walk.

ML Tasks: **Gradient Descent**

Definition

Gradient descent finds the bottom of the loss bowl by repeating one move: measure the slope of the loss where you stand, take a small step in the downhill direction.

Analogy

You stand on a hill in thick fog. You cannot see the valley, but you can feel which way the ground tilts under your feet. Step downhill, feel again, step again. You reach the bottom without ever seeing the map.

1. Start with any parameters, for example $w = 0$, $b = 0$.
2. Compute the loss and its **slope** with respect to each parameter (the **gradient**).
3. Move each parameter a little **against** its slope.
4. Repeat until the loss stops shrinking.

For the line $\hat{y}_i = w x_i + b$ with errors $e_i = \hat{y}_i - y_i$, the slopes of the MSE are

$$\frac{\partial L}{\partial w} = \frac{1}{n} \sum_{i=1}^n 2 e_i x_i \qquad \frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^n 2 e_i$$

- ▶ $\partial L / \partial w$ (“the derivative of L with respect to w ”): how fast the loss grows if w grows a little. Positive: w is too big. Negative: too small.
- ▶ The pair of slopes is the **gradient**. It points uphill, so we step the other way.
- ▶ Reading the formula: an error e_i on a sample with a large x_i pulls the slope harder than the same error on a small x_i .

Skipped on purpose

Where the $2 e_i x_i$ comes from (the chain rule) is Day 11 for lines and Day 15 for networks.
Today: trust the formula and use it.

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w} \qquad b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

- ▶ $\leftarrow$: “becomes”. The new value replaces the old one, like `w = w - ...` in Python.
- ▶ α (alpha): the **learning rate**, the step size. A small positive number you choose, like 0.1 or 0.01.
- ▶ The minus sign is the whole trick: the gradient points uphill, so subtracting it walks downhill.

In words

If the loss goes up when w goes up, decrease w . If the loss goes down when w goes up, increase w . Move more when the slope is steep, less when it is flat.

Data $(1, 3), (2, 5), (3, 7)$; start $w = 0, b = 0$; learning rate $\alpha = 0.1$.

$$\hat{y} = (0, 0, 0)$$

$$e = \hat{y} - y = (-3, -5, -7) \quad L = \frac{9+25}{3}$$

$$\frac{\partial L}{\partial w} = \frac{2}{3}((-3)(1) + (-5)(2) + (-7)(3)) = \frac{2}{3}(-34) = -22.67$$

$$\frac{\partial L}{\partial b} = \frac{2}{3}(-3 - 5 - 7) = \frac{2}{3}(-15) = -10$$

$$w \leftarrow 0 - 0.1 \cdot (-22.67) = 2.267$$

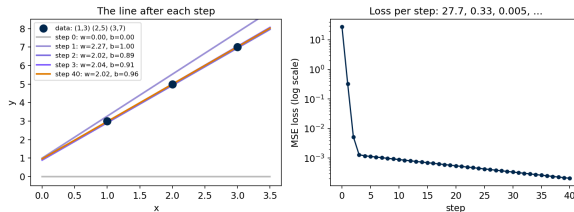
$$b \leftarrow 0 - 0.1 \cdot (-10) = 1.0$$

- ▶ Both slopes are negative: the line is too low, so both parameters go **up**.
- ▶ After one step the line is $\hat{y} = 2.267x + 1.0$, and the loss drops from 27.67 to 0.33.
- ▶ The true answer is $w = 2, b = 1$. One step overshoot w a little; the next steps fix it.

The Steps, in a Table and a Picture

step	w	b	loss
0	0.000	0.000	27.667
1	2.267	1.000	0.332
2	2.018	0.893	0.005
3	2.044	0.908	0.001
50	2.013	0.970	0.0001
200	2.000	0.999	0.0000

The big gains come first; the last decimals take hundreds of small steps. That is normal.



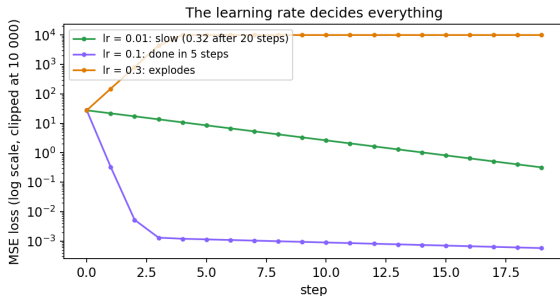
Source: the three-point example, 40 steps with $\alpha = 0.1$; scripts/day08_ml.figures.py.

```
import numpy as np
x = np.array([1., 2., 3.]); y = np.array([3., 5., 7.])
w, b, lr = 0.0, 0.0, 0.1

for step in range(200):
    y_hat = w * x + b                # predict
    e = y_hat - y                    # errors
    grad_w = np.mean(2 * e * x)      # slope of the loss for w
    grad_b = np.mean(2 * e)          # slope of the loss for b
    w, b = w - lr * grad_w, b - lr * grad_b  # step downhill
print(round(w, 3), round(b, 3))      # 2.0 0.999
```

- The loop body is the four moves of the recipe: predict, error, gradient, update. Every trainer in Weeks 3 to 6 has the same four lines inside.

The Learning Rate Decides Everything



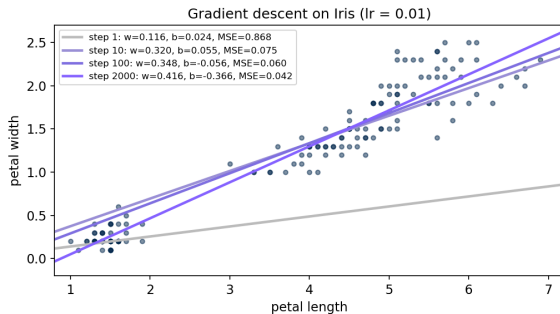
Source: the three-point example, 20 steps for each α ; scripts/day08_ml_figures.py.

- ▶ $\alpha = 0.01$: every step is tiny; after 20 steps the loss is still 0.32.
- ▶ $\alpha = 0.1$: done in about 5 steps.
- ▶ $\alpha = 0.3$: each step jumps over the bottom and lands higher on the other side. The loss **explodes**: 27, 150, 812, ...

The common mistake

A loss that grows or prints `nan` is almost always a learning rate that is too big. Divide it by 10 and retry.

The Same Loop on 150 Real Flowers



Source: Iris, Kaggle uciml/iris; the ten-line loop with $\alpha = 0.01$; scripts/day08_ml_figures.py.

Replace x and y by the petal length and petal width columns. Nothing else changes.

step	w	b	MSE
1	0.116	0.024	0.868
10	0.320	0.055	0.075
100	0.348	-0.056	0.060
2000	0.416	-0.366	0.042

The loop arrives at the same 0.416 and -0.36 we saw earlier. Nobody typed those numbers: the data chose them.

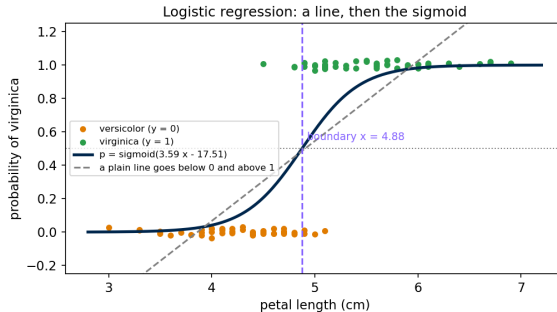
```
from sklearn.linear_model import LinearRegression
X = df[["PetalLengthCm"]].values      # 2-D: 150 rows, 1 column
y = df["PetalWidthCm"].values         # 1-D: 150 values

model = LinearRegression()            # a Day 04 object
model.fit(X, y)                       # chooses w and b
print(model.coef_, model.intercept_) # [0.416] -0.363
print(model.predict([[4.0]]))         # [1.30]
```

- ▶ fit stores the parameters in the object (`coef_` = the weights, `intercept_` = the bias), exactly Day 04's pattern.
- ▶ X must be 2-D even with one feature: hence the double brackets.
- ▶ scikit-learn solves this particular problem with a formula instead of a loop, so it is exact and instant. The formula (Day 11) only exists for lines; everything bigger uses the loop.

ML Tasks: **Logistic Regression**

Why a Plain Line Fails for Yes/No



Source: Iris, versicolor and virginica only, Kaggle uciml/iris;
`sklearn.linear_model.LogisticRegression`; `scripts/day08_ml_figures.py`.

Task: is this flower a virginica? Target $y = 1$ (yes) or 0 (no), feature: petal length.

- ▶ A straight line (dashed) predicts -0.2 at 3 cm and 1.2 at 7 cm. A “probability” below 0 or above 1 means nothing.
- ▶ What we want is the S-shaped curve: near 0 on the left, near 1 on the right, crossing 0.5 where the species mix.

The fix

Keep the line, then squash its output into $(0, 1)$ with one extra function.

The Sigmoid, With Numbers

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- z : any number, from $-\infty$ to $+\infty$.
 $e \approx 2.718$. $\sigma(z)$: always between 0 and 1.

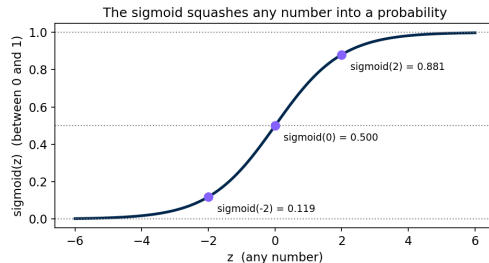
Worked example

$$\sigma(0) = 1/(1 + 1) = 0.5$$

$$\sigma(2) = 1/(1 + e^{-2}) = 1/1.135 = 0.881$$

$$\sigma(-2) = 1/(1 + e^2) = 1/8.389 = 0.119$$

Big positive z : almost 1. Big negative: almost 0. $z = 0$: exactly the middle.



Source: $\sigma(z)$ evaluated on 300 points; `scripts/day08_ml_figures.py`.

Definition

Logistic regression predicts the probability of “yes” as $p = \sigma(wx + b)$, and answers yes when $p \geq 0.5$. Despite its name, it is a **classification** model.

- ▶ $p \geq 0.5$ exactly when $w x + b \geq 0$: the **decision boundary** is $x = -b/w$. On the two Iris species, $w = 3.59$ and $b = -17.51$, so the boundary is $17.51/3.59 = 4.88$ cm.

Worked example

Petal length 4.0: $z = 3.59 \cdot 4.0 - 17.51 = -3.15$, $p = \sigma(-3.15) = 0.041$: versicolor.

Petal length 5.5: $z = 3.59 \cdot 5.5 - 17.51 = 2.24$, $p = \sigma(2.24) = 0.904$: virginica.

The rule is right for 93 of the 100 flowers (accuracy 0.93).

Skipped on purpose: w and b are found by gradient descent too, with a different loss (log loss, not MSE). Tomorrow names it; Day 11 shows the steps.

```
from sklearn.linear_model import LogisticRegression
two = df[df["Species"] != "Iris-setosa"]           # 100 flowers
X = two[["PetalLengthCm"]].values
y = (two["Species"] == "Iris-virginica").astype(int) # 1 = yes

clf = LogisticRegression().fit(X, y)
print(clf.coef_, clf.intercept_)                  # [[3.59]] [-17.51]
print(clf.predict_proba([[4.0], [5.5]]))[:, 1])    # [0.041 0.904]
print(clf.predict([[4.0], [5.5]]))                 # [0 1]
```

- ▶ Same fit / predict pattern as the line. `predict_proba` gives the probability of each class; column 1 is “yes”.
- ▶ With three species scikit-learn fits one line per species and picks the largest probability. Same idea, three sets of dials.

ML Tasks: Olympiad Practice

Task

You get `train.csv` (3,000 games) and `test.csv` (600 games without the target). Columns: Metacritic score (0 to 100), Positive and Negative (review counts), Estimated owners (a range such as "20000 - 50000"), Genres, and the target Price in dollars. Predict Price for every test game and hand in a CSV with one column, Price. Scoring: mean absolute error, $MAE = \frac{1}{n} \sum |\hat{y}_i - y_i|$. Full points below 6.6, 50 points below 7.0, 40 below 8, 30 below 9, else 10.

- ▶ First question: **what is the target and what kind is it?** A price is a number: **regression**, so a linear regression is the first model to try.
- ▶ Second question: **which columns are already numbers?** Three are; the owners range needs the midpoint of the two numbers; genres are text (Day 12).

Adapted from the ONIA (Romanian AI Olympiad) official problem-proposal example "Predicting Game Prices on the Steam Platform", Olimpiada-AI, 2024/25; data from the Kaggle Steam Games data set.

Recognise: numeric target, so a linear regression; MAE as the score, so check it on held-out rows before submitting.

```
cols = ["Metacritic score", "Positive", "Negative"]
lo = train["Estimated owners"].str.split(" - ").str[0].astype(float)
hi = train["Estimated owners"].str.split(" - ").str[1].astype(float)
train["owners"] = (lo + hi) / 2 # midpoint of the range
X, y = train[cols + ["owners"]], train["Price"]

model = LinearRegression().fit(X[:2400], y[:2400]) # 80% to fit
pred = model.predict(X[2400:]) # 20% to check
print(np.mean(np.abs(pred - y[2400:]))) # MAE 8.2
```

- **Check:** predicting the mean price for everyone gives MAE 8.65; this line gives 8.20 (40 points). Adding the release year as a fifth column drops it to 7.49; the official solution's line, with the genres one-hot encoded (Day 12), reaches 6.89, and a random forest (Day 10) 6.57, full marks.
- **Read the weights:** `model.coef_[0]` is 0.23: each Metacritic point is worth about 23 cents. The dials are readable, which trees are not.

ML Tasks: **Limitations & What Comes Next**

- ▶ **A line cannot bend.** If the true relation is a curve (price against age of a car, height against age of a child), the best line is still wrong everywhere. Monday's trees and KNN can bend.
- ▶ **Squares love outliers.** One game priced at \$200 pulls the whole line up, because its error is squared. Day 09 shows when MAE is the safer loss.
- ▶ **Features on different scales.** Review counts in the thousands next to scores from 0 to 100 make gradient descent crawl or explode. Day 11 scales the columns first.
- ▶ **Training error is not the score.** A loss of 0.042 on the 150 flowers you trained on says nothing about the 151st. Tomorrow splits the data to find out.

The common mistake

Reporting the loss on the training rows and calling it the model's accuracy. Always keep rows the model has never seen for the check.

- ▶ **Tomorrow (Day 09), Loss Functions and Metrics:** MSE against MAE, accuracy against F1, and why the loss the model minimises is not always the score the judge uses.
- ▶ **Also Day 09:** underfitting and overfitting, the train/test split, and regularization: a penalty that keeps the weights small.
- ▶ **Monday (Day 10):** models that are not lines: KNN, SVMs, decision trees, random forests, and how to tune them.
- ▶ **Days 11 to 13, self-study:** linear and logistic regression in depth, trees and feature engineering, then cross-validation, PCA and feature importance.

ML Tasks: **Summary**

- ▶ **AI $\supset$ ML $\supset$ DL:** AI is the task, ML learns the rules from data, DL does it with many-layered neural networks.
- ▶ **Task types:** regression (a number), classification (a category), clustering and PCA (no labels). Name the type before writing code.
- ▶ **A model is a function with dials:** $\hat{y} = mx + b$ has a slope m and an intercept b ; with many features, weights $\mathbf{w}$ and a bias b .
- ▶ **The loss** (MSE) turns “how good is this line” into one number, and draws a bowl over the parameters.
- ▶ **Gradient descent** walks to the bottom of the bowl: predict, error, gradient, step against it, repeat; the learning rate sets the step.
- ▶ **scikit-learn** does it in two calls, `fit` and `predict`; **logistic regression** adds a sigmoid to turn the line into a probability for yes/no.

ML Tasks: **References & Resources**

- [1] Fisher, R. A. (1936). *Iris* data set, Kaggle copy `uciml/iris`. Used for every real-data figure and code example. <https://www.kaggle.com/datasets/uciml/iris>
- [2] Olimpiada-AI (2024/25). *Predicting Game Prices on the Steam Platform*, the ONIA problem-proposal example (task, evaluator and reference solutions). Adapted for the practice task. <https://github.com/Olimpiada-AI/Propunere-probleme-si-solutii>
- [3] Fronkon Games. *Steam Games Dataset*, Kaggle. The data behind the practice task. <https://www.kaggle.com/datasets/fronkongames/steam-games-dataset>
- [4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12. LinearRegression, LogisticRegression, KMeans, PCA. <https://scikit-learn.org>
- [5] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., chapters 1 and 4. The traditional-programming versus ML contrast and the gradient-descent presentation follow it.
- [6] All plots: `slides-builder/scripts/day08_ml_figures.py` (matplotlib, real data or the three-point example; no decorative images).